Android Auto

Complete Deep Learning Notes

From Phone to Car Display — Everything You Need to Know

by **Swapnil Patil**

Android Auto is Google's platform for projecting Android app experiences from a connected phone onto a car's infotainment display. Unlike Android Automotive OS (AAOS) — where the app runs natively on the car's hardware — Android Auto runs on your phone and streams the UI to the car screen over USB or wireless. This document covers every major concept required to build, test, and publish Android Auto apps — from setup and architecture to templates, media integration, voice control, security, and testing with the Desktop Head Unit (DHU).

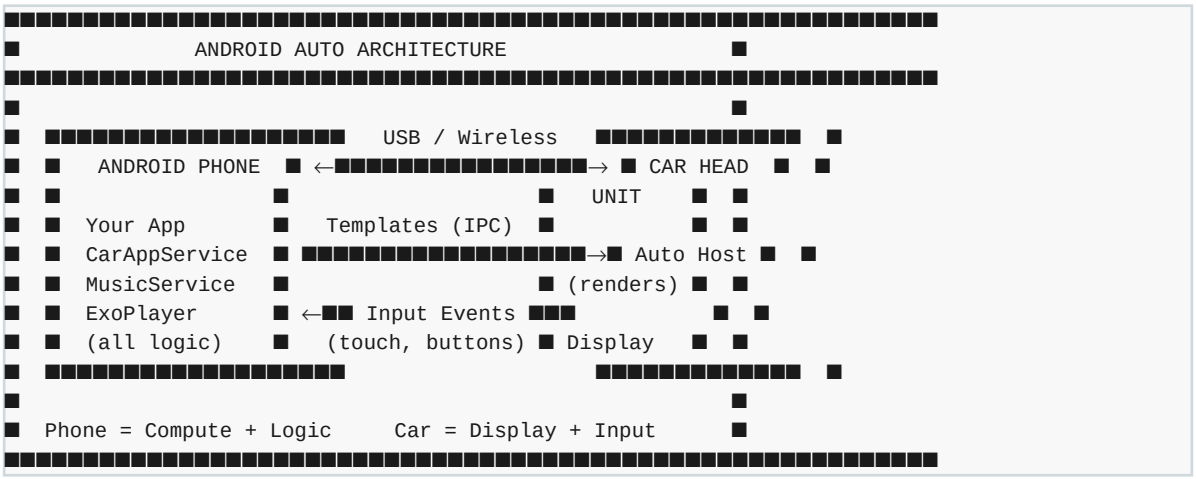
This guide is designed for developers who already understand general Android development and want to specifically master Android Auto. Every topic includes a plain-language explanation of what it is, why it matters for car apps, and detailed code examples with inline comments.

TOPIC 1 Android Auto Overview

What is it? Android Auto is a platform from Google that allows your Android app to run on a phone and project a simplified, driver-safe UI onto a car's infotainment display. The phone does all the computation; the car screen just shows the result. The two devices communicate over USB (wired) or Wi-Fi (wireless Android Auto, Android 11+). The car's infotainment unit runs a Host app (provided by Google or the OEM) that renders the UI templates your app sends over the connection.

Why do we use it? Android Auto has a much larger reach than AAOS because it works with any Android phone on thousands of car models — you don't need a car with built-in AAOS hardware. As of 2024, Android Auto is available in over 500 car models worldwide. If you want to reach the broadest possible audience of car users today, Android Auto is the right target. AAOS will eventually be more common, but Android Auto is what most drivers currently use.

How Android Auto Projection Works



Key Components

Component	Where it runs	Role
Your App	Phone	All business logic, media playback, templates
CarAppService	Phone	Entry point — car host binds to this
Android Auto Host	Car head unit	Renders templates, sends input events back
Car screen	Car hardware	Displays projected UI, receives user touch
MediaSession	Phone	Exposes playback controls to the car host

What Android Auto Can Run

- **Media apps** — music, podcasts, audiobooks (most common)
- **Navigation apps** — turn-by-turn directions with map rendering
- **Messaging apps** — read/reply messages via voice
- **POI apps** — points of interest, parking, EV charging

Key Point: Android Auto only supports apps in specific categories. You cannot build a generic productivity or social app for Android Auto — only the approved categories above.

TOPIC 2 Android Auto vs Android Automotive OS (AAOS)

What is it? Android Auto and AAOS both use the Car App Library and look similar in code, but they are fundamentally different platforms. Android Auto is a projection system — your app runs on the phone, the car just shows the picture. AAOS is a full embedded OS — your app runs on the car's own computer without any phone. Understanding this distinction is critical for deciding which platform to target and how to structure your project.

Aspect	Android Auto	Android Automotive OS
App runs on	Android phone	Car's own CPU/ECU
Phone required?	Yes — always connected	No — completely standalone
Car hardware needed?	Any car with AA support (500+ models)	AAOS car (Volvo, Polestar, etc.)
Internet access	Phone's SIM / WiFi	Car's built-in SIM / WiFi
Gradle module	app/ (uses artifact: app)	automotive/ (uses: app-automotive)
Car API access	None (no CarPropertyManager)	Full OEM APIs available
Vehicle sensors	Not accessible	Speed, RPM, fuel, HVAC, etc.
OEM integration	Limited to screen projection	Deep native integration
Market reach	Very large (billions of Android users)	Growing (newer AAOS cars)
Update mechanism	User updates phone app	OTA update or Play Store on car
Background execution	On phone — normal Android rules	On car — AAOS power management
Testing	Desktop Head Unit (DHU) or real car	AAOS emulator or real AAOS car
minSdk	API 23 minimum (recommended 26)	API 29 minimum (required)

Which Platform Should You Target?

- **Android Auto** — if you want maximum reach today, simpler integration, no vehicle data needed
- **AAOS** — if you need vehicle sensor data, deep OEM integration, or a standalone car-first experience
- **Both** — the Car App Library makes it easy to support both from one codebase with separate modules

Tip: For a new app targeting car users in 2024-2025, support both: use the shared Car App Library for common UI logic, with the app/ module for Android Auto and automotive/ for AAOS. Most Car App Library code is identical between the two.

TOPIC 3 Phone Module Setup — app/ Module

What is it? The app/ module is the Android Auto phone module. It is a standard Android application module that targets a regular Android phone. Your CarAppService, Session, and Screen classes live here (or in a shared module). The critical difference from the AAOS automotive/ module is the Gradle dependency: you use the 'app' artifact (not 'app-automotive') from the Car App Library. This artifact includes classes for phone-side rendering and communication with the car host.

Why do we use it? The app/ module is what gets installed on the user's phone via the Play Store. When the user connects their phone to a car that supports Android Auto, the car's Android Auto host discovers and connects to your CarAppService in this module. Without the correct artifact (app vs app-automotive), the car host won't recognize your service and the app won't appear in the car.

app/build.gradle.kts

```
plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
}

android {
    compileSdk = 36

    defaultConfig {
        applicationId = "com.swapnil.smart.aaos"  // same as automotive module
        minSdk    = 23     // Android Auto works from API 23
                           // (recommended: 26 for full feature support)
        targetSdk = 36
        versionCode = 1
        versionName = "1.0"
    }

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
    }
    kotlinOptions { jvmTarget = "11" }
}

dependencies {
    // KEY DIFFERENCE: use 'app' NOT 'app-automotive'
    implementation(libs.androidx.car.app)              // phone artifact

    // Same media dependencies as automotive module
    implementation(libs.media3.exoplayer)
    implementation(libs.androidx.media)
    implementation(libs.androidx.lifecycle.viewmodel)
    implementation(libs.androidx.lifecycle.livedata)
}
```

Artifact Difference — Critical

Module	Artifact	Class included	When to use
app/	app	CarAppActivity (phone projection)	Android Auto (phone module)
automotive/	app-automotive	CarAppActivity (native AAOS)	AAOS (car module)

Root build.gradle.kts — Multi-Module Setup

```
// Root build.gradle.kts
plugins {
    alias(libs.plugins.android.application) apply false
```

```
    alias(libs.plugins.kotlin.android)      apply false
}

// settings.gradle.kts — include both modules
include(":app")        // Android Auto phone module
include(":automotive") // AAOS car module
```

Warning: Do NOT add the automotive module as a dependency of the app module (or vice versa). They are separate applications published independently to the Play Store. Shared code should go into a separate :shared library module.

TOPIC 4 Manifest Setup for Android Auto

What is it? The AndroidManifest.xml for an Android Auto app has specific declarations that differ from both a regular phone app and an AAOS app. You must declare the car application metadata, link the automotive_app_desc.xml category file, and declare your CarAppService with the correct intent filter. Incorrect manifest setup is the #1 reason Android Auto apps don't appear in the car launcher.

Why do we use it? The Android Auto host on the car scans all installed apps on the connected phone, looking for apps with the CarAppService intent filter and the correct metadata. Without the meta-data declaration, your app is invisible to the car host. Without the correct category in automotive_app_desc.xml, your app won't appear in the right section of the car launcher (media, navigation, etc.).

app/AndroidManifest.xml — Complete Setup

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <!-- Required for background music playback -->
    <uses-permission android:name="android.permission.FOREGROUND_SERVICE"/>
    <uses-permission android:name="android.permission.FOREGROUND_SERVICE_MEDIA_PLAYBACK"/>
    <uses-permission android:name="android.permission.INTERNET"/>

    <!-- NOTE: No CAR_SPEED, CAR_ENGINE etc. — Android Auto has NO vehicle sensor access -->

    <application
        android:label="Smart Auto"
        android:icon="@mipmap/ic_launcher">

        <!-- CRITICAL: links to automotive_app_desc.xml -->
        <!-- This is how the Android Auto host discovers your app category -->
        <meta-data
            android:name="com.google.android.gms.car.application"
            android:resource="@xml/automotive_app_desc"/>

        <!-- Minimum Car App API level (1 = widest compatibility) -->
        <meta-data
            android:name="androidx.car.app.minCarApiLevel"
            android:value="1"/>

        <!-- CarAppActivity: required for Android Auto phone projection -->
        <activity
            android:name="androidx.car.app.activity.CarAppActivity"
            android:exported="true"
            android:launchMode="singleTask"
            android:theme="@android:style/Theme.DeviceDefault.NoActionBar">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>

        <!-- CarAppService: the Android Auto host binds to this -->
        <service
            android:name=".car.SmartCarAppService"
            android:exported="true">
            <intent-filter>
                <!-- Required: identifies this as a Car App service -->
                <action android:name="androidx.car.app.CarAppService"/>
                <!-- Required: declares media category -->
                <category android:name="androidx.car.app.category.MEDIA"/>
            </intent-filter>
        </service>
```

```
        <!-- Music service: Android Auto uses this for media controls -->
        <service
            android:name=".media.SmartMusicService"
            android:exported="true">
            <intent-filter>
                <action android:name="android.media.browse.MediaBrowserService"/>
                <action android:name="android.media.action.MEDIA_PLAY_FROM_SEARCH"/>
            </intent-filter>
        </service>

        <!-- MediaButtonReceiver: handles steering wheel media buttons -->
        <receiver android:name="androidx.media.session.MediaButtonReceiver"
                  android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MEDIA_BUTTON"/>
            </intent-filter>
        </receiver>

    </application>
</manifest>
```

automotive_app_desc.xml — Android Auto vs AAOS

```
<!-- app/src/main/res/xml/automotive_app_desc.xml -->
<!-- For Android Auto (phone module) -->
<automotiveApp>
    <uses name="media" />
    <!-- Other options: "navigation", "notification", "video" -->
</automotiveApp>

<!-- AAOS uses the SAME file format in automotive/ module -->
<!-- The Car App Library handles both from the same XML -->
```

Note: The automotive_app_desc.xml file must be placed in res/xml/ of the app/ module for Android Auto, and in res/xml/ of the automotive/ module for AAOS. They can have the same content if your app supports the same categories on both platforms.

TOPIC 5 Android Auto App Categories

What is it? Android Auto only allows specific categories of apps to run on the car display. These categories are declared in automotive_app_desc.xml and the CarAppService intent filter. Each category gets different UI templates, permissions, and placement in the car launcher. Submitting an app to the wrong category causes Play Store rejection.

Why do we use it? Google restricts what kind of apps can run on Android Auto to protect driver safety. Only categories that have a legitimate, driver-safe use case in a car are allowed. Each category also gets purpose-built templates — a navigation app gets map rendering support, a media app gets playback controls, a messaging app gets TTS read-aloud support. Choosing the right category unlocks the correct set of templates and APIs.

Category	Intent Filter Category	Use Case	Key Templates
Media	androidx.car.app.category.MEDIA	Music, podcast, audiobook, radio	ListTemplate, PaneTemplate
Navigation	androidx.car.app.category.NAVIGATION	Turn-by-turn directions, maps	NavigationTemplate + SurfaceContainer
Point of Interest	androidx.car.app.category.POI	Parking, EV charging, restaurants	PlaceListMapTemplate
IOT	androidx.car.app.category.IOT	Smart devices, remote control	ListTemplate, MessageTemplate

Media Category — Most Common

```
<!-- automotive_app_desc.xml -->
<automotiveApp>
    <uses name="media" />
</automotiveApp>

<!-- AndroidManifest.xml -->
<service android:name=".car.SmartCarAppService" android:exported="true">
    <intent-filter>
        <action android:name="androidx.car.app.CarAppService"/>
        <category android:name="androidx.car.app.category.MEDIA"/>  <!-- media -->
    </intent-filter>
</service>
```

Navigation Category — Special Surface Access

```
<!-- automotive_app_desc.xml -->
<automotiveApp>
    <uses name="navigation" />
</automotiveApp>

<!-- AndroidManifest.xml -->
<service android:name=".car.SmartCarAppService" android:exported="true">
    <intent-filter>
        <action android:name="androidx.car.app.CarAppService"/>
        <category android:name="androidx.car.app.category.NAVIGATION"/> <!-- nav -->
    </intent-filter>
</service>

<!-- Navigation apps get access to SurfaceContainer for drawing a map -->
<!-- Only navigation category apps can use NavigationTemplate -->
```

Key Point: SmartAAOS is a media app — it uses androidx.car.app.category.MEDIA. Do not mix categories (e.g., listing both media and navigation) — this causes Play Store rejection.

TOPIC 6 CarAppService & Session on Android Auto

What is it? CarAppService and Session work the same way in Android Auto as in AAOS — CarAppService is the entry point the car host binds to, and Session manages one active car display connection. The code is identical between the app/ and automotive/ modules because both use the same Car App Library API. The difference is under the hood: in Android Auto the Session runs on the phone and the templates are projected to the car; in AAOS the Session runs natively on the car's hardware.

Why do we use it? Because the Car App Library abstracts the platform, you can share CarAppService and Screen implementations between Android Auto and AAOS. In SmartAAOS, the code in SmartCarAppService.kt, SmartSession.kt, and all Screen files would work with no changes if moved to the app/ module. This is the main benefit of the Car App Library architecture.

CarAppService — Identical Code for Both Platforms

```
// This code works in BOTH app/ (Android Auto) AND automotive/ (AAOS)
class SmartCarAppService : CarAppService() {

    override fun createHostValidator(): HostValidator {
        // Android Auto: validates the Android Auto host app on the car
        // AAOS: validates the OEM car host
        // In production use HostValidator.Builder() with known host signatures
        return HostValidator.ALLOW_ALL_HOSTS_VALIDATOR  // development only
    }

    override fun onCreateSession(): Session = SmartSession()

    override fun onDestroy() {
        super.onDestroy()
        CarViewModelStore.clear()
    }
}
```

Session Lifecycle Differences

```
// Android Auto Session lifecycle events
class SmartSession : Session() {

    override fun onCreateScreen(intent: Intent): Screen {
        // Called when Android Auto connects to your phone app
        // The car screen is now showing your app's UI
        return HomeScreen(carContext)
    }

    override fun onNewIntent(intent: Intent) {
        // Called for voice commands or deep links while session is active
        handleVoiceIntent(intent)
    }

    // NOTE: No onStart/onStop in Session — lifecycle is managed by the car host
    // The session ends when the user disconnects the phone from the car
}
```

Event	Trigger	What happens
onCreateSession()	Car host binds to CarAppService	Session created, screen prepared
onCreateScreen()	First screen requested by car host	Return your HomeScreen
onNewIntent()	New intent arrives (voice, deep link)	Handle voice commands
Session active	Phone connected to car	All screens visible, interactive
Session ends	Phone disconnected / user exits app	Service may be unbound

Tip: In Android Auto, the Session can also end when the car is turned off or the user switches to another app. Always handle cleanup in CarAppService.onDestroy() rather than assuming the session will end cleanly.

TOPIC 7 Android Auto Connection Lifecycle

What is it? The Android Auto connection lifecycle describes all the states a phone-to-car connection goes through: from initial USB/wireless pairing, through the car host discovering and binding your app, to the session becoming active and the car screen showing your UI, all the way to disconnection. Understanding this lifecycle is critical for managing resources correctly — starting and stopping services, requesting audio focus, and cleaning up connections.

Why do we use it? Resource leaks and crashes in Android Auto apps almost always happen because developers don't properly handle lifecycle events — leaving ExoPlayer running after disconnect, not releasing MediaBrowserCompat connections, or holding audio focus after the session ends. Knowing exactly when each callback fires helps you write clean, reliable code.

```
Full Android Auto Connection Lifecycle:

1. User connects phone to car (USB or Wireless)
   ■■ Android Auto app on car launches and discovers AA-compatible apps on phone

2. Car host binds to CarAppService on phone
   ■■ CarAppService.onCreateHostValidator() called
   ■■ CarAppService.onCreateSession() called → Session object created

3. Car host requests first screen
   ■■ Session.onCreateScreen(intent) called → HomeScreen returned
   ■■ HomeScreen.onGetTemplate() called → ListTemplate built and sent to car
   ■■ Car screen renders the template — user sees your app

4. Session is ACTIVE
   ■■ User taps/speaks → car host sends input event back to phone
   ■■ setOnClickListener / MediaControllerCompat callbacks fire on phone
   ■■ Screen.invalidate() → new template sent to car → screen updates

5. New intent arrives (voice command)
   ■■ Session.onNewIntent(intent) called
   ■■ handleVoiceIntent() → playFromSearch() → PlayerScreen pushed

6. User exits app or disconnects phone
   ■■ ScreenManager cleared
   ■■ Session destroyed
   ■■ CarAppService.onDestroy() called → clean up ViewModels, release resources
   ■■ MusicService continues running in foreground (phone-side) if music is playing
```

Note: The music service (SmartMusicService) is NOT stopped when the Android Auto session ends — it continues running on the phone as a foreground service. The user can still see the media notification on their phone and control playback from there.

TOPIC 8 Template-Based UI on Android Auto

What is it? Android Auto uses the exact same Car App Library templates as AAOS — ListTemplate, PaneTemplate, MessageTemplate, NavigationTemplate, etc. The template code you write is platform-agnostic; the Car App Library serializes it and sends it to the car host, which renders it. On Android Auto, the car head unit's Android Auto app does the rendering. On AAOS, the OEM's template host does it. Your code doesn't change.

Why do we use it? This is the greatest strength of the Car App Library — write once, run on both platforms. The template system guarantees driver safety by enforcing strict limits on text, actions, and screen complexity. You cannot bypass these limits even if you wanted to — the library validates your template structure before serializing it.

Full Template Catalogue

Template	Category	Description	Key Limit
ListTemplate	All	Scrollable list with rows and images	Max 6 items (API 1)
PaneTemplate	All	Detail pane with rows + action buttons	Max 2 actions
MessageTemplate	All	Simple title + message text	Max 2 actions
GridTemplate	All	Grid of image tiles (album art grid)	Max 8 items
SearchTemplate	All	Search input field + results list	Max 6 results
NavigationTemplate	Nav only	Full-screen map + turn-by-turn overlay	Nav category only
PlaceListMapTemplate	POI/Nav	Map + list of nearby places	Max 6 places
RoutePreviewTemplate	Nav only	Route overview before starting navigation	Max 3 routes
SignInTemplate	All (API4)	Login form for account-based apps	API 4+
LongMessageTemplate	All (API2)	Scrollable long text (terms, etc.)	API 2+

CarIcon — Images in Templates

All images in templates must be wrapped in a CarIcon. You can create CarIcons from a Bitmap (e.g., album art loaded with Glide), a vector drawable resource, or a standard system icon. The car host scales them appropriately for the display.

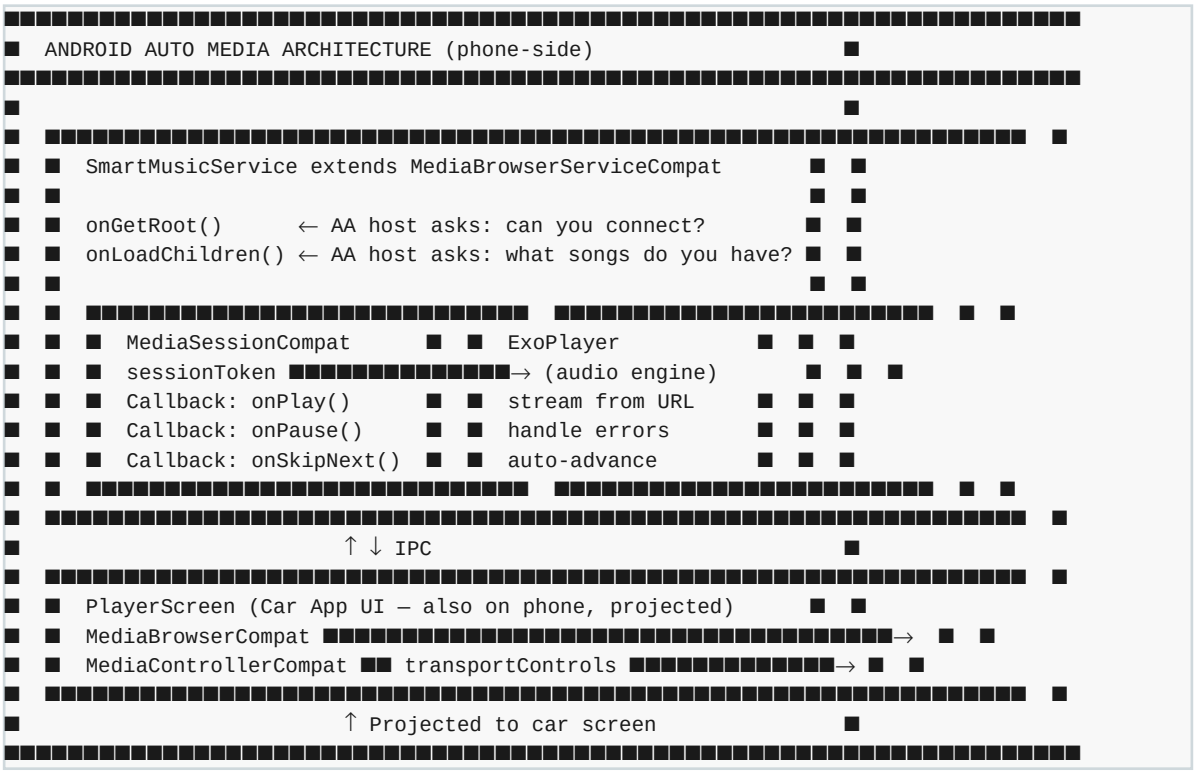
```
// From a Bitmap (album art loaded from URL)
val carIcon = CarIcon.Builder(
    IconCompat.createWithBitmap(bitmap)
).build()

// From a drawable resource
val carIcon = CarIcon.Builder(
    IconCompat.createWithResource(carContext, R.drawable.ic_music_note)
).build()

// From a standard Car App icon
val carIcon = CarIcon.APP_ICON  // built-in: app, alert, back, compose, error

// Use in a Row
Row.Builder()
    .setTitle(song.title)
    .setImage(carIcon, Row.IMAGE_TYPE_LARGE)  // LARGE = big album art thumbnail
    .build()
```

TOPIC 9 Media Apps on Android Auto

What is it? A media app on Android Auto is built around three layers: (1) MediaBrowserServiceCompat — the service that exposes your media catalogue and hosts the MediaSession; (2) MediaSession — the control hub that receives play/pause/skip commands from the car; (3) ExoPlayer (or MediaPlayer) — the actual audio engine on the phone. The Android Auto host connects to your MediaBrowserServiceCompat using MediaBrowserCompat, discovers your song list via onLoadChildren(), and controls playback via MediaSession callbacks.

Why do we use it? Android Auto's media controls (on-screen buttons, steering wheel buttons, voice commands, and the Android Auto 'media' UI section) all work through the MediaSession and MediaBrowserServiceCompat APIs. Without implementing these, the Android Auto host cannot control your app's playback and your app will not appear in the car's media picker. The media architecture is mandatory for any music or audio app on Android Auto.

Android Auto Media Architecture



Media Queue — Providing Song List to Android Auto

In addition to onLoadChildren(), Android Auto's media UI shows a playback queue. You should set the queue on MediaSession so the car's native media player screen can display 'Up Next' properly.

```
// Set the playback queue on MediaSession
val queue = MusicData.songs.mapIndexed { index, song ->
    val desc = MediaDescriptionCompat.Builder()
        .setMediaId(song.id)
        .setTitle(song.title)
        .setSubtitle(song.artist)
        .build()
    MediaSessionCompat.QueueItem(desc, index.toLong())
}
session.setQueue(queue)
session.setQueueTitle("Smart AAOS Playlist")

// Also set active queue item (currently playing song)
session.setPlaybackState(
    PlaybackStateCompat.Builder()
        .setState(STATE_PLAYING, position, 1.0f)
```

```
        .setActiveQueueItemId(currentIndex.toLong())  // highlight current song
        .build()
)
```

TOPIC 10 Navigation Apps on Android Auto

What is it? Navigation apps on Android Auto are a special category that gets access to a Surface — a raw drawing canvas provided by the car host where you can render a map using Canvas2D or OpenGL. On top of this map surface, you overlay a NavigationTemplate with turn-by-turn instructions, ETA, distance, and action buttons. Navigation apps must declare the NAVIGATION category and implement NavigationSession instead of a plain Session.

Why do we use it? The navigation category is special because rendering a map requires custom drawing — you cannot represent a dynamic map with Car App Library templates alone. The SurfaceContainer API bridges this gap: it gives your app a guaranteed portion of the screen to draw on (the map area), while the car host renders the overlaid template UI (turn-by-turn card, action buttons) on top. This is how Google Maps, Waze, and other navigation apps work on Android Auto.

NavigationSession — Session Subclass for Nav Apps

```
// Navigation apps extend NavigationSession instead of Session
class SmartNavigationSession : NavigationSession() {

    override fun onCreateScreen(intent: Intent): Screen {
        return MapScreen(carContext)
    }

    // Called when the car host provides a Surface to draw the map on
    override fun onCreateMap(surfaceContainer: SurfaceContainer): MapController {
        return SmartMapController(carContext, surfaceContainer)
    }
}
```

SurfaceContainer — Drawing the Map

```
class SmartMapController(
    private val carContext: CarContext,
    private val surfaceContainer: SurfaceContainer
) : MapController() {

    override fun onSurfaceAvailable(surfaceContainer: SurfaceContainer) {
        // Surface is ready — start drawing your map
        val surface = surfaceContainer.surface
        val canvas = surface.lockCanvas(null)
        // Draw map tiles, route, current location...
        canvas.drawBitmap(mapTileBitmap, 0f, 0f, null)
        surface.unlockCanvasAndPost(canvas)
    }

    override fun onSurfaceDestroyed(surfaceContainer: SurfaceContainer) {
        // Surface gone — stop drawing, release resources
    }

    override fun onVisibleAreaChanged(visibleArea: Rect) {
        // Car host tells you which area of the surface is visible
        // (the rest may be covered by the template overlay)
        redrawMap(visibleArea)
    }
}
```

NavigationTemplate

```
// NavigationTemplate overlays on top of the map surface
NavigationTemplate.Builder()
    .setNavigationInfo(
        RoutingInfo.Builder()
            .setCurrentStep(
```

```
            Step.Builder("Turn right onto MG Road")
                .setManeuver(Maneuver.Builder(Maneuver.TYPE_TURN_RIGHT).build())
                .setRoad("MG Road")
                .build()
        )
        .setDistanceToStep(Distance.create(0.5, Distance.UNIT_KILOMETERS))
        .build()
)
.setDestinationTravelEstimate(
    TravelEstimate.Builder(
        Distance.create(12.5, Distance.UNIT_KILOMETERS),
        DateTimeWithZone.create(/* ETA */ ...)
    ).build()
)
.setActionStrip(ActionStrip.Builder().addAction(muteAction).build())
.build()
```

TOPIC 11 Android Auto Permissions

What is it? Android Auto apps use standard Android permissions declared in the phone module's manifest. There are NO special car permissions like CAR_SPEED or CAR_ENGINE_DETAILED — those are exclusive to AAOS where the app runs on the car hardware and can access vehicle sensors. Android Auto apps only access data available on the phone (storage, internet, microphone, etc.). Vehicle data (speed, RPM) is simply not accessible from Android Auto.

Why do we use it? Understanding the permission boundary between Android Auto and AAOS prevents a common developer mistake: assuming Android Auto can access vehicle sensors. It cannot. If your app needs real vehicle data (speed for driving restrictions, fuel level for alerts), you must target AAOS. Android Auto apps use only phone-side data sources.

Permission	Platform	Used For
FOREGROUND_SERVICE	Both	Keep music service alive in background
FOREGROUND_SERVICE_MEDIA_PLAYBACK	Both	Foreground service type for audio
INTERNET	Both	Stream audio from URLs
READ_MEDIA_AUDIO	Both	Access local music files
RECORD_AUDIO	Auto only	Voice input (messaging apps)
android.car.permission.CAR_SPEED	AAOS only	Read vehicle speed from car sensor
android.car.permission.CAR_ENGINE_DETAILED	AAOS only	Read RPM, engine temp from car
android.car.permission.CAR_ENERGY	AAOS only	Read fuel level, battery level
android.car.permission.CAR_INFO	AAOS only	Read VIN, model, year from car

Minimal Permissions for Android Auto Media App

```
<!-- app/AndroidManifest.xml — Android Auto media app -->
<uses-permission android:name="android.permission.FOREGROUND_SERVICE"/>
<uses-permission android:name="android.permission.FOREGROUND_SERVICE_MEDIA_PLAYBACK"/>
<uses-permission android:name="android.permission.INTERNET"/>

<!-- Optional: local file playback -->
<uses-permission android:name="android.permission.READ_MEDIA_AUDIO"/>

<!-- NOT included (AAOS only): -->
<!-- android.car.permission.CAR_SPEED -->
<!-- android.car.permission.CAR_ENGINE_DETAILED -->
<!-- android.car.permission.CAR_ENERGY -->
```

Warning: Adding CAR_SPEED or other CAR_* permissions to the app/ module's manifest will NOT grant vehicle sensor access — they will simply be ignored on a phone. Worse, it may cause Play Store reviewers to flag your app as requesting inappropriate permissions.

TOPIC 12 HostValidator & Security

What is it? HostValidator is a Car App Library class that controls which car hosts are allowed to connect to your CarAppService. A 'host' is the Android Auto app on the car head unit, or the OEM template host on AAOS. Since your CarAppService is exported (visible to other apps), a malicious app could theoretically bind to it and control your media service. HostValidator prevents this by requiring the connecting host to match a known certificate.

Why do we use it? Security: a CarAppService that allows any host to connect could be bound by a malicious app on the same device, gaining access to your media controls and potentially user data. In production, you should restrict connections to known, trusted car hosts (Google's Android Auto host, Samsung DeX, known OEM hosts) using their signing certificates. ALLOW_ALL_HOSTS_VALIDATOR is only for development and testing.

Development vs Production HostValidator

```
// DEVELOPMENT ONLY — allows any host to connect (includes your DHU)
override fun createHostValidator(): HostValidator {
    return HostValidator.ALLOW_ALL_HOSTS_VALIDATOR
}

// PRODUCTION — restrict to known trusted hosts
override fun createHostValidator(): HostValidator {
    return HostValidator.Builder(applicationContext)
        // Allow Google's Android Auto host
        .addAllowedHost(
            "com.google.android.projection.gearhead",  // AA package name
            HostValidator.ANDROID_AUTO_DIGEST            // Google's cert digest
        )
        // Allow Android Auto Simulator (for CI testing)
        .addAllowedHost(
            "com.google.android.autosimulator",
            HostValidator.ANDROID_AUTO_SIMULATOR_DIGEST
        )
        .build()
}
```

Known Host Package Names

Host	Package Name	Context
Android Auto (production)	com.google.android.projection.gearhead	Real car, production AA
Android Auto Simulator	com.google.android.autosimulator	Testing, DHU
Android Auto (older versions)	com.google.android.apps.auto	Legacy devices
AAOS template host	Varies by OEM	AAOS cars

Note: HostValidator.ANDROID_AUTO_DIGEST and HostValidator.ANDROID_AUTO_SIMULATOR_DIGEST are pre-defined constants in the Car App Library. You don't need to hardcode certificate hashes manually.

TOPIC 13 Testing with Desktop Head Unit (DHU)

What is it? The Desktop Head Unit (DHU) is a tool provided in the Android SDK that simulates an Android Auto car head unit on your development computer. It connects to the Android Auto app on your test phone over USB (or via the Android emulator), projects your Car App Library UI onto a desktop window, and lets you interact with it using your mouse and keyboard — simulating the car touchscreen, steering wheel buttons, and voice commands. It is the primary testing tool for Android Auto development.

Why do we use it? Real car testing is expensive, slow, and often impractical during development. The DHU lets you test every aspect of your Android Auto app — templates, navigation, media controls, voice commands, screen sizes, day/night mode — entirely on your laptop. It is essential for iterating quickly on your UI and catching template violations before they cause crashes in a real car.

DHU Setup — Step by Step

```
Step 1: Install DHU via Android SDK Manager
    Android Studio → SDK Manager → SDK Tools
    ✓ Check: Android Auto Desktop Head Unit emulator
    Path: $ANDROID_HOME/extras/google/auto/

Step 2: Enable Developer Mode in Android Auto on your phone
    Phone → Android Auto app → tap version number 10 times
    → Developer settings unlocked → Enable 'Unknown sources'

Step 3: Start the DHU server on your phone
    Android Auto app → Developer settings → Start head unit server
    (Phone now listens for DHU connection)

Step 4: Connect phone to computer via USB
    adb forward tcp:5277 tcp:5277

Step 5: Launch the DHU on your computer
    cd $ANDROID_HOME/extras/google/auto/
    ./desktop-head-unit          (macOS/Linux)
    desktop-head-unit.exe        (Windows)
```

DHU Controls & Keyboard Shortcuts

Action	Keyboard / Method	What it tests
Tap	Mouse click	Touch input on templates
Back button	Esc	ScreenManager.pop()
Voice input	Ctrl+M (microphone)	MEDIA_PLAY_FROM_SEARCH
Media play/pause	Space bar	MediaSession onPlay/onPause
Media next	N	MediaSession onSkipToNext
Media previous	P	MediaSession onSkipToPrevious
Day mode	D	Light theme rendering
Night mode	Ctrl+N	Dark theme rendering
Wide screen	W	Test wide car display layout
Simulate steering wheel btns	S + arrow	Hardware media button handling

Tip: Run your app in debug mode (adb debugging enabled) while using the DHU. Logcat filters for 'SmartAAOS' will show all your Log.d() calls — this is the fastest way to debug template rendering issues.

TOPIC 14 Android Auto Emulator Testing

What is it? Android Auto can also be tested using the Android Emulator (AVD) without a physical phone. You create an AVD with the Android Auto system image or use the Automotive OS system image to test AAOS. The emulator approach is useful for CI/CD pipelines or when you don't have a physical Android device with the Android Auto app installed. For AAOS, there is a dedicated 'Automotive' hardware profile in Android Studio.

Why do we use it? Physical device + DHU setup requires a compatible phone with Android Auto installed, which may not always be available in a CI environment. The emulator allows automated UI tests to run headlessly. The AAOS emulator (automotive hardware profile) also lets you test AAOS-specific behavior (vehicle sensors, system integrations) without owning an AAOS car.

Creating an AAOS Emulator (Android Studio)

```
Step 1: Open AVD Manager
    Android Studio → Tools → Device Manager → Create Virtual Device

Step 2: Select Hardware Profile
    Category: Automotive
    Profile: Automotive (1024p landscape)  ← standard AAOS display
             OR: Automotive Portrait        ← portrait AAOS display

Step 3: Select System Image
    Tab: Other Images
    Release: Android 12L or 13 Automotive (AAOS)
    ABI: x86_64
    Note: Download if not already present (~3GB)

Step 4: Configure AVD
    Name: AAOS_Test
    RAM: 4096 MB minimum
    Storage: 8192 MB
    Graphics: Hardware - GLES 2.0

Step 5: Launch and test
    Run → Select AAOS_Test as deployment target
    App installs and launches in the AAOS emulator
```

Android Auto Emulator (phone-based)

```
For Android Auto (phone projection testing) using an emulator:

Step 1: Create a standard phone AVD (Pixel 6, API 33+)

Step 2: Install Android Auto APK on the emulator
    adb install android_auto.apk

Step 3: Use the DHU to connect to the emulator
    adb -e forward tcp:5277 tcp:5277    (-e = emulator)
    ./desktop-head-unit

Step 4: In the emulator, start head unit server
    Android Auto app → Developer settings → Start head unit server
```

Note: The AAOS emulator is the most accurate way to test AAOS-specific features. However, it does not simulate real vehicle sensor data (speed, RPM) — you need to mock these in your VehicleRepository as SmartAAOS does with VehicleDataService.

TOPIC 15 AAOS vs Android Auto — Code Differences

What is it? While the Car App Library allows most code to be shared between AAOS and Android Auto, there are key differences in manifest setup, Gradle dependencies, permissions, and available APIs. Understanding exactly what changes (and what stays the same) is critical when supporting both platforms from one codebase.

Side-by-Side Comparison

What	Android Auto (app/)	AAOS (automotive/)
Gradle artifact	androidx.car.app:app	androidx.car.app:app-automotive
minSdk	23 (recommended: 26)	29 (required)
uses-feature	None required	android.hardware.type.automotive REQUIRED
Vehicle permissions	None — no vehicle sensor access	CAR_SPEED, CAR_ENGINE_DETAILED, etc.
CarAppActivity	Declared in manifest (phone launcher)	Declared in manifest (car launcher)
HostValidator	Validate Android Auto host certificate	Validate OEM host certificate
VehicleDataService	Not used — no vehicle sensor IPC	AIDL service for sensor data
CarPropertyManager	Not available	Available for real OEM sensor data
MediaBrowserService	Required — AA host uses it for media	Required — same API
Testing tool	DHU (Desktop Head Unit)	AAOS Emulator or AAOS car
Play Store listing	Regular app listing (appears for AA users)	Automotive section listing

What is Identical in Both

- CarAppService class and its lifecycle methods
- Session class and onCreateScreen / onNewIntent
- All Screen subclasses (HomeScreen, PlayerScreen, etc.)
- All Template classes (ListTemplate, PaneTemplate, etc.)
- ScreenManager navigation (push, pop, back)
- MediaBrowserServiceCompat and MediaSession APIs
- ExoPlayer integration and audio focus management
- Foreground service and MediaStyle notification
- Voice control via MEDIA_PLAY_FROM_SEARCH
- All Car App Library UI components (Row, Action, CarIcon, etc.)

Shared Module Pattern — Best Practice

When supporting both platforms, extract shared code into a :shared module to avoid duplication:

```
// settings.gradle.kts
include(":app")        // Android Auto
include(":automotive") // AAOS
include(":shared")     // shared CarAppService, Screens, MusicService

// app/build.gradle.kts
dependencies {
    implementation(project(":shared"))
    implementation(libs.androidx.car.app)              // Auto artifact
}

// automotive/build.gradle.kts
```

```
dependencies {
    implementation(project(":shared"))
    implementation(libs.androidx.car.app.automotive)   // AAOS artifact
}
```

TOPIC 16 Publishing Android Auto Apps to Play Store

What is it? Publishing an Android Auto app to the Play Store requires additional steps beyond a standard Android app submission. Google reviews all Android Auto apps manually for driver safety compliance before they appear in the Android Auto section. Your app must pass the Driver Distraction Guidelines review — which checks template usage, driving mode restrictions, voice support, and overall UX safety.

Why do we use it? Unlike regular Android apps that go live automatically after review, Android Auto apps require explicit approval from Google's Android Auto team. This process can take 1-3 weeks for the initial review. Understanding what reviewers check allows you to build compliant apps from the start and avoid rejection cycles.

Publishing Checklist

Step	Action	Details
1	Test with DHU	Test all screens, voice commands, driving restrictions
2	Check template compliance	Verify max items, actions, text lines per template
3	Verify driving restrictions	No complex taps/inputs allowed at speed > 5 km/h
4	Test voice commands	MEDIA_PLAY_FROM_SEARCH must work with common queries
5	Check HostValidator	Use production HostValidator (not ALLOW_ALL) for release
6	Sign the APK	Use release signing key, not debug key
7	Update Play Console	App content → Android Auto section → fill required fields
8	Submit for review	Google manually reviews Android Auto apps
9	Address review feedback	May request video demo of the app in a real car or DHU
10	Publish	App appears in Play Store automotive section after approval

Driver Distraction Guidelines — Key Rules

- **No text entry while driving** — disable text input fields when speed > 0
- **Max 6 list items** at Car App API 1 (more allowed at higher API levels)
- **Max 2 actions** on PaneTemplate — no more than 2 decision points per screen
- **Voice must work** — onPlayFromSearch() must return a result for common music queries
- **No custom drawing** unless navigation category with SurfaceContainer
- **App must be usable with one hand** — no multi-touch gestures
- **Content must update without user interaction** — no manual refresh required

Warning: Apps that add interaction triggers while driving (e.g., allowing song changes at 60 km/h) will fail Google's safety review. Always check vehicle speed or use the isCarMoving flag before enabling any non-essential interaction.

Key Point: For AAOS apps, the review process goes through the car OEM's certification process in addition to (or instead of) Google's Play Store review. Contact the OEM's developer program for AAOS-specific certification requirements.

Complete Android Auto Architecture Summary